Repository: https://github.com/sunitazenith07-lang/ClubBudget-Open-Source-Evaluation

RewardBook: Rewards Rules & Repeat-Transaction Alerts

Track: Fintech loyalty | **Time:** 24 hours | **Language:** Python 3.10+

Scenario

A loyalty product awards points for eligible purchases. Promotions may add a bonus or cap. Purchases can later be refunded; duplicate transaction events must not award points twice. Operations also need transparent warnings for possible same-account, same-day, same-amount repeats.

This is a fictional offline service inspired by common rewards products. It is not any company's internal code and uses no real financial/customer data.

Required behaviour

1. Calculate points using integer arithmetic: base_points = floor(amount_paise / 10000) *points_per_100_rupees; bonus_points = floor(base_points* (100 + bonus_percent) / 100); apply optional cap_points last.

2. Invalid or negative amounts/rules must not generate points. A cap of zero means zero points.

3. Only captured purchases earn points. Declined, pending and unknown events must not.

4. Process each event_id once so replaying purchase/refund events cannot alter balances twice.

5. Reverse points proportionally for refunds using integer arithmetic, never reverse more than awarded and never allow a negative balance.

6. Write an auditable ledger entry for each accepted purchase/refund event.

7. Flag possible repeats by account, date and amount using the supplied threshold. Alerts are for human review only; never block a transaction or automatically accuse a customer.

8. Handle empty input, missing identifiers, unknown transactions and boundary values safely.

Setup

Run: `python -m pip install -r requirements-dev.txt`, then `python -m pytest -q`, then `python -m main`.

Work to submit

- Read the source and run the starter tests before editing.
- Use the requirements and tests as the acceptance contract. Trace failures to their root causes, fix the underlying behaviour, and avoid hard-coding sample output.
- Add regression tests and document at least five edge cases in `TESTING.md`.

- Submit one pull request to the original repository with the fixes, tests and a brief explanation. Do not merge it.
- Include `REWARD_RULE_NOTES.md` with one worked points calculation, one partial-refund example and three edge cases.

Mandatory problem-solving documentation — all three files required

Submit these three completed Markdown files at the repository root in your pull request. They are evaluated; code alone is not a complete submission.

- `approach.md` — record your starting test result, the code/data flow you explored, key hypotheses, experiments you ran, implementation strategy, and why you chose it.
- `challenges.md` — **COMPULSORY**. Describe concrete obstacles or confusing behaviours, what you tried, evidence from tests/logs, how you resolved them, and any unresolved limitations. If you did not get stuck, state that honestly and describe the hardest edge case you deliberately tested; do not invent a struggle.
- `optimizations.md` — describe optimizations considered or implemented, why you chose or rejected them, correctness/readability/performance trade-offs, and future improvements. Separate measured improvements from ideas you did not measure.

Keep the notes concise, specific and evidence-based. Describe your actual work, tests and technical decisions; do not provide private hidden chain-of-thought or a stream-of-consciousness transcript. Generic or copied text does not demonstrate your process. Each file is worth 5 marks. A missing or empty file receives 0/5 for that component and makes the documentation requirement incomplete.

Optional stretch work

Choose at most one: generate a CSV account statement; make the repeat threshold configurable; or add a rewards summary by category.

Testing and GitHub Actions

The untouched starter is expected to have some failing visible tests because behaviours are incomplete. The exact test count is not fixed. GitHub Actions runs on pushes and pull-request updates and publishes test counts. A red check before fixes is expected; make the visible tests pass without deleting, skipping, weakening or bypassing them. Do not alter the workflow to evade failures.

Individual-work policy

AI chatbots, AI coding assistants, generated patches/tests and third-party solution-writing are prohibited. Normal non-AI documentation is allowed. Do not upload assessment code or data to external services. You must explain your changes and make a small live modification if asked. A walkthrough supports integrity review; automated AI detectors are not definitive proof of use.

Rubric — 100 marks

- Understanding code and data flow: 10
- Points calculation and business-rule correctness: 20

- Idempotency, refunds, ledger integrity and repeat alerts: 25
- Testing and root-cause analysis: 15
- Code quality and maintainability: 5
- Required problem-solving documentation: 15 (approach.md: 5, challenges.md: 5, optimizations.md: 5)
- Pull request, rewards notes and live explanation: 10

The three process documents are assessed for specific, honest, evidence-based technical reflection, not writing style.